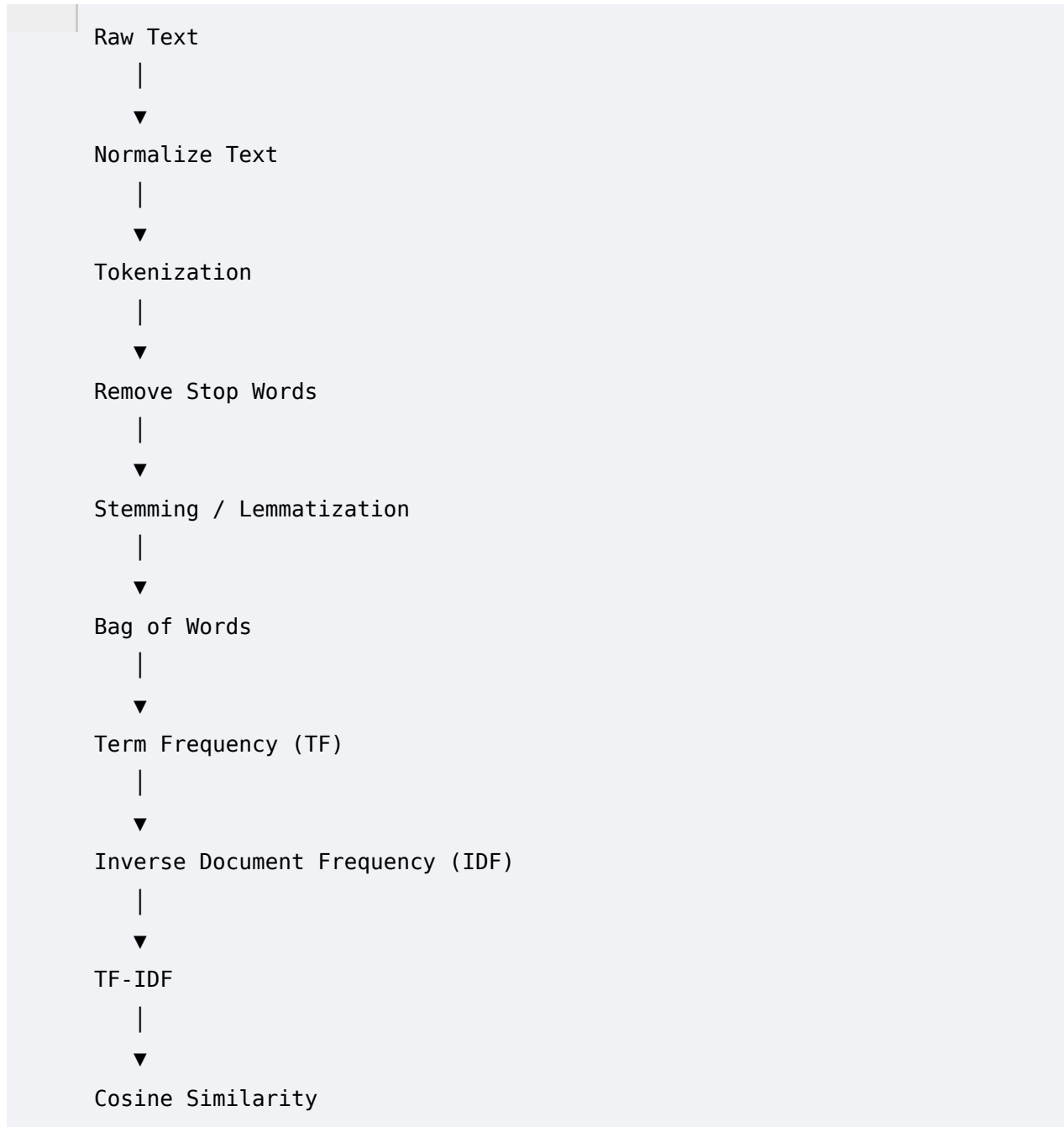


Phase 1 - Traditional NLP Summary

This is the complete pipeline followed by most traditional NLP systems.



1. NormalizeText()

PURPOSE

Clean the raw text into a standard format.

Without normalization

```
Hello
HELLO
```

```
hello
Hello!
```

Computer thinks these are different.

After normalization

```
hello
hello
hello
hello
```

Now they become identical.

COMMON OPERATIONS

- Convert to lowercase
- Remove punctuation
- Remove extra spaces
- Remove special characters
- (Optional) Remove accents

Example

Input

```
"Hello!!!   World."
```

Output

```
hello world
```

Real-world use

Every NLP system starts with normalization.

2. Tokenization()

PURPOSE

Split text into individual words (tokens).

Input

```
I love machine learning
```

Output

```
I  
love  
machine  
learning
```

or

```
[  
  "I",  
  "love",  
  "machine",  
  "learning"  
]
```

Why?

Computers process words individually.

Real-world use

Without tokenization

```
"I love NLP"
```

is one string.

With tokenization

```
"I"  
"love"  
"NLP"
```

Now each word can be analyzed.

3. RemoveStopWords()

PURPOSE

Remove common words that add little meaning.

Example stop words

```
the  
is
```

```
am
are
was
were
a
an
of
to
in
on
```

Example

Input

```
I am learning NLP in C#
```

Tokens

```
I
am
learning
NLP
in
C#
```

After removing stop words

```
learning
NLP
C#
```

Why?

These words appear everywhere.

They don't help distinguish documents.

4. Stemming()

PURPOSE

Reduce words to their root form by chopping off endings.

Example

```
playing
played
plays
```

↓

```
play
```

More examples

```
running → run
studies → studi
fishing → fish
```

Notice

```
studies
```

↓

```
studi
```

This isn't a real English word.

That's why stemming is fast but inaccurate.

Advantages

- Fast
- Simple

Disadvantages

- Can produce incorrect words
-

5. Lemmatization()

PURPOSE

Convert words into their dictionary form.

Example

```
running
ran
runs
```



run

More examples

better → good
children → child
mice → mouse

Unlike stemming

studies



study

which is a real word.

Advantages

- More accurate
- Produces valid words

Disadvantages

- Slower
- Needs a dictionary/model

Comparison

Word	Stemmin g	Lemmatiz ation
studies	studi	study
running	run	run
better	better	good
children	children	child

6. BagOfWords()

PURPOSE

Convert text into numbers.

Machine learning cannot understand text.

It understands numbers.

Example

Vocabulary

```
apple  
banana  
orange
```

Sentence

```
apple banana apple
```

Count words

```
apple = 2  
banana = 1  
orange = 0
```

Vector

```
[2,1,0]
```

Meaning

Position matters.

```
Index 0 → apple  
  
Index 1 → banana  
  
Index 2 → orange
```

Limitation

Bag of Words ignores word order.

These produce identical vectors:

Dog bites man

Man bites dog

7. CalculateTF()

PURPOSE

Measure how important a word is inside one document.

Formula

$$\text{TF} = \frac{\text{Word Count}}{\text{Total Words}}$$

Example

Document

apple apple banana orange

Total words

4

Counts

apple = 2
banana =1
orange =1

TF

apple
 $2/4 = 0.5$
banana
 $1/4 = 0.25$

orange

$1/4 = 0.25$

Meaning

Higher TF

↓

Word appears more often.

8. CalculateIDF()

PURPOSE

Measure how unique a word is across all documents.

Formula

$IDF = \log(\text{Total Documents} / \text{Documents containing word})$

Example

Three documents

Doc1

apple banana

Doc2

apple orange

Doc3

apple mango

Word

apple

Appears

3 documents

IDF

$$\log(3/3)$$

$$=0$$

Meaning

Not important.

Word

mango

Appears

1 document

IDF

$$\log(3/1)$$

$$\approx 1.10$$

Meaning

Very informative.

Rule

Common words

↓

Low IDF

Rare words

↓

High IDF

9. Calculate TFIDF()

PURPOSE

Combine local importance (TF) and global uniqueness (IDF).

Formula

$$\text{TF-IDF} = \text{TF} \times \text{IDF}$$

Example

TF

```
apple =0.5  
banana=0.25
```

IDF

```
apple=0  
  
banana=1.1
```

TF-IDF

```
apple  
  
0.5×0  
  
=0  
  
banana  
  
0.25×1.1  
  
=0.275
```

Even though apple appears more often,
banana becomes more important because it's rarer across the corpus.

Why not use TF only?

Words like

```
the  
is  
this
```

would get high scores because they occur frequently.

Why not use IDF only?

IDF ignores how many times the word appears in the current document.

TF-IDF balances both.

10. CalculateCosineSimilarity()

PURPOSE

Measure how similar two documents are.

Instead of comparing text,

we compare TF-IDF vectors.

Formula

$$\text{Similarity} = \frac{\mathbf{A} \cdot \mathbf{B}}{|\mathbf{A}| \times |\mathbf{B}|}$$

Where:

- $\mathbf{A} \cdot \mathbf{B}$ is the dot product of the vectors.
- $|\mathbf{A}|$ and $|\mathbf{B}|$ are the magnitudes (lengths) of the vectors.

The result ranges from **0 to 1** for non-negative TF-IDF vectors:

- **1.0** → Exactly the same direction (very similar)
 - **0.0** → Completely different (no shared terms)
-

Example

Vocabulary

```
apple  
banana  
orange
```

Document A

```
apple banana
```

Vector

```
[1,1,0]
```

Document B

apple orange

Vector

[1,0,1]

Cosine similarity ≈ 0.5 , indicating the documents share some content but are not identical.

Real-world uses

- Search engines rank documents by similarity to a query.
- FAQ and chatbot systems find the closest matching question.
- Document recommendation systems.
- Duplicate document detection.
- Resume matching.

Complete Example

Suppose the document is:

"The cat is running in the garden."

Step	Output
Normalize Text()	the cat is running in the garden
Tokenizati on()	[the, cat, is, running, in, the, garden]

RemoveStopWords()	[cat, running, garden]
Stemming()	[cat, run, garden]
Lemmatization()	[cat, run, garden]
BagOfWords()	[cat=1, run=1, garden=1]
CalculateTF()	{cat=0.33, run=0.33, garden=0.33}
CalculateIDF()	Computed from the entire document collection
CalculateTFIDF()	{cat=0.42, run=0.15, garden=0.68}

	<i>(example values)</i>
Calculate Cosine Similarity()	Used to compare this document with another document or query

WHAT YOU'VE COMPLETED

You have now covered the core building blocks of traditional NLP:

-  Text preprocessing (Normalization, Tokenization, Stop Word Removal, Stemming, Lemmatization)
-  Feature extraction (Bag of Words)
-  Statistical weighting (TF, IDF, TF-IDF)
-  Similarity measurement (Cosine Similarity)

This foundation prepares you well for the next phase of NLP, where you'll move from counting words to learning semantic meaning using **Word Embeddings (Word2Vec, GloVe, FastText)**, followed by **Transformers, Vector Search, RAG**, and **LLM-based applications**.